BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.

4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search (UCS) chooses the next node using only the cumulative path cost from the start node, g(n). It does not know where the goal is, so it expands evenly in all directions, like growing circles around the start.

A* Search chooses the next node using f(n) = g(n) + h(n). Here g(n) is the actual cost so far and h(n) is a heuristic estimate of the remaining cost to the goal. The heuristic steers the search toward the goal, so A* expands far fewer nodes than UCS and still finds the optimal path, as long as h(n) is admissible

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

A heuristic is admissible if it never overestimates the true cost to the goal, so h(n) <= h*(n) for every n. On a grid with 4-way movement and a cost of 1 per step, moving from (x1, y1) to (x2, y2) needs at least |x1 - x2| + |y1 - y2| moves, even with no obstacles. Walls can only make the real path longer. Manhattan distance is therefore the minimum possible cost and never overestimates, which makes it admissible.

If the heuristic is not admissible (h(n) > h*(n)), A* is no longer guaranteed to find the optimal path. A node on the true shortest path can get an inflated f-value. A* then expands other nodes first and may reach the goal through a longer path. The result is a suboptimal solution.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. With diagonal movement, Manhattan distance overestimates the true cost. For example, going from (0,0) to (1,1) takes 1 diagonal move, so h* = 1. Manhattan distance gives |1| + |1| = 2, and 2 > 1 violates admissibility.

The metric to switch to is Chebyshev distance: h(n) = max(|x1 - x2|, |y1 - y2|). This is admissible when diagonal and straight moves both cost 1.

If diagonal moves cost sqrt(2) and straight moves cost 1, use Octile distance: h(n) = (dx + dy) + (sqrt(2) - 2) * min(dx, dy), where dx = |x1 - x2| and dy = |y1 - y2|.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Using only the distance to the closest food ignores all the other food, so it is a weak heuristic. A stronger one is the Minimum Spanning Tree (MST) heuristic:

h(n) = (cost of the MST over all remaining food) + (Manhattan distance from the agent to the nearest food)

- Treat the remaining food positions as nodes in a complete graph, with Manhattan distance as the edge weights.
- Compute the MST of those food nodes using Prim's or Kruskal's algorithm.
- Add the distance from the agent's current position to the closest food pellet.

It is admissible because any path that visits all the food must connect all of those points. The MST is the cheapest way to connect them, so its cost is at most the length of the shortest tour (the Traveling Salesperson route). The heuristic therefore never overestimates.

It is also more informed than the nearest-food distance, because it takes into account how spread out all the remaining food is. This means A* expands fewer nodes

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING